

Haskell 復習ノート : type, newtype, data, Maybe, Just, Functor, Applicative, Monad

Hiroki Funashima

Thursday, May 21, 2026



目次

1	このノートの目的	5
2	type : 型の別名	6
2.1	type C = Complex Double	6
2.2	type は新しい型を作らない	7
3	newtype の基本	8
3.1	基本形	8
3.2	型名と値コンストラクタ名は同じでなくてもよい	8
4	newtype の制約	10
4.1	1 コンストラクタ・1 フィールド	10
4.2	タプルにまとめれば newtype にできる	10
5	data があるのに newtype がある理由	11
5.1	型安全性だけなら data でもよい	11
5.2	newtype は実行時コストのないラッパー	11
5.3	type, newtype, data の比較	11
6	data の基本	12
6.1	複数フィールドを持つ型	12
7	data と直積型・直和型	13
7.1	Point は直積型	13
7.2	複数コンストラクタは直和型	13
7.3	各コンストラクタの中は直積, コンストラクタ同士は直和	14
8	はガードではなくコンストラクタの選択肢	15
9	Maybe と Just	16
9.1	Maybe の定義	16
9.2	Just は値を一段持ち上げる	16
9.3	Maybe Int は Int Nothing そのものではない	16
9.4	パターンマッチによる取り出し	17
10	実践例 : 3 次元 bra, ket, operator	18
10.1	内部表現	18
10.2	newtype による意味の境界	18
10.3	Operator とユニタリ性	19

10.4	Mat3 を Matrix ではなく (Vec3, Vec3, Vec3) にした理由	19
11	deriving のまとめ	21
11.1	deriving とは何か	21
11.2	最頻出の 3 つ: Eq, Ord, Show	22
11.3	列挙型でよく使う Enum と Bounded	25
11.4	Read	26
11.5	Ix	26
11.6	GHC 拡張で導出できるもの	27
11.7	newtype と GeneralizedNewtypeDeriving	28
11.8	DerivingStrategies	29
11.9	Ket, Bra, Operator で何を derive するべきか	30
11.10	まとめ	30
12	newtype と data における演算定義の違い	32
12.1	問題意識	32
12.2	type の場合: 元の型そのもの	32
12.3	newtype の場合: 新しい型だが演算を流用しやすい	33
12.4	deriving newtype による明示	33
12.5	data の場合: 演算の意味を明示的に定義する	34
12.6	なぜ newtype では流用しやすいのか	34
12.7	ただし, newtype でも意味的な妥当性が重要	35
12.8	量子ベクトルの例で考える	35
12.9	比較表	36
12.10	まとめ	36
13	この後半で扱う流れ	38
14	Maybe / Either から Functor, Applicative, Monad へ	39
14.1	出発点: Just 5 :: Maybe Int は一段持ち上がっている	39
14.2	Maybe の直感: 失敗するかもしれない値	40
14.3	Either の直感: 失敗理由を持つ値	40
14.4	Maybe と Either の関係	41
15	Functor: 中身に関数を適用する	43
15.1	問題: Maybe Int の中の Int に関数を適用したい	43
15.2	fmap の型	44
15.3	Maybe に対する fmap の動き	44
15.4	演算子 <\$>	44
15.5	Either に対する fmap	45

15.6	Functor の直感	45
16	Applicative : 文脈付きの関数を文脈付きの値に適用する	47
16.1	Functor だけでは足りない場合	47
16.2	Applicative の基本関数	47
16.3	Maybe に対する Applicative	48
16.4	二引数関数への適用	48
16.5	pure による持ち上げ	49
16.6	Either に対する Applicative	49
16.7	Functor と Applicative の違い	50
17	Monad : 結果に応じて次の計算を選ぶ	51
17.1	Applicative では表しにくい場合	51
17.2	<code>>>=</code> の型	51
17.3	Maybe に対する bind	52
17.4	fmap と <code>>>=</code> の違い	52
17.5	例 : 安全な計算をつなぐ	53
17.6	do 記法	54
17.7	do 記法と命令型プログラムの見た目	54
17.8	Either に対する Monad	55
18	Functor, Applicative, Monad の関係	57
18.1	型クラスの包含関係	57
18.2	段階的な理解	57
18.3	具体例で比較する	57
18.4	Maybe で見る 3 段階	58
18.5	Either で見る 3 段階	59
19	Monad を特別視しすぎない	60
19.1	Monad は「文脈付き計算の接続」である	60
19.2	return と pure	60
19.3	一段持ち上げる, 一段の中で変換する, 次の計算へ渡す	60
19.4	設計上の意味	61
20	まとめ	62
20.1	type	62
20.2	newtype	62
20.3	data	62
20.4	Maybe と Just	63
20.5	全体像	63

1 このノートの目的

このノートでは、Haskell の型定義に関する基本事項を整理する。中心となるのは次の構文である。

```
type    -- 型の別名を与える
newtype -- 既存の型を包み、新しい型として扱う
data    -- 代数的データ型を定義する
```

特に、次の観点を重視する。

- 型名と値コンストラクタの違い
- `type` が `alias` であること
- `newtype` が「型安全な軽量ラッパー」であること
- `data` が直積型・直和型を表すこと
- `Maybe` と `Just` が「一段持ち上げた値」を表すこと

2 type : 型の別名

2.1 type C = Complex Double

次の定義を考える.

```
import Data.Complex

type C = Complex Double
```

これは,

Complex Double という型に C という別名を付ける

という意味である.

したがって,

```
C
```

と

```
Complex Double
```

は, Haskell コンパイラから見ると同じ型である.

例えば次は問題ない.

```
z1 :: C
z1 = 1.0 :+ 2.0

z2 :: Complex Double
z2 = z1
```

ここで `:+` は `Data.Complex` による複素数の構成子である.

```
1.0 :+ 2.0
```

は数学的には

$$1.0 + 2.0i$$

を表す.

したがって,

```
type C = Complex Double
```

は,

C を「倍精度浮動小数点数 Double を実部・虚部に持つ複素数型」の別名にする

という定義である.

2.2 type は新しい型を作らない

重要なのは，type は新しい型を作っていないという点である．

```
type C = Complex Double
```

は alias であり，次のような newtype とは異なる．

```
newtype C = C (Complex Double)
```

前者は単なる別名である．後者は Complex Double を包んだ新しい型である．

3 newtype の基本

3.1 基本形

代表例として、次の定義を考える。

```
newtype UserId = UserId Int
```

これは、

UserId という新しい型を定義し、その値は Int を 1 つ包む値コンストラクタ UserId によって作られる

という意味である。

この定義では、同じ名前 UserId が 2 回出てくる。

```
newtype UserId = UserId Int
--      ^^^^^^      ^^^^^^  ^^^
--      型名        値コンストラクタ 中身の型
```

左辺の UserId は型名である。右辺の UserId は値コンストラクタである。

値コンストラクタとしての型は次である。

```
UserId :: Int -> UserId
```

つまり、右辺の UserId は

Int を受け取って UserId 型の値を作る関数

のように使える。

```
uid :: UserId
uid = UserId 42
```

ここで、

```
UserId 42 :: UserId
```

である。

3.2 型名と値コンストラクタ名は同じでなくてもよい

次のように書くこともできる。

```
newtype UserId = MkUserId Int
```

この場合、

```
MkUserId :: Int -> UserId
```


である.

値を作るときは次のようになる.

```
uid :: UserId
uid = MkUserId 42
```

取り出すときはパターンマッチを使う.

```
getUserId :: UserId -> Int
getUserId (MkUserId n) = n
```

したがって,

```
newtype UserId = UserId Int
```

で同じ名前を使うのは, 慣習的な書き方である.

4 newtype の制約

4.1 1 コンストラクタ・1 フィールド

newtype は、必ず

- 値コンストラクタを 1 つだけ持つ
- その値コンストラクタのフィールドを 1 つだけ持つ

という制約を持つ。

したがって、これは許される。

```
newtype UserId = UserId Int

newtype Name = Name String

newtype Meter = Meter Double
```

しかし、これは許されない。

```
newtype Point = Point Double Double
```

なぜなら、値コンストラクタ Point が

```
Point :: Double -> Double -> Point
```

のように 2 つのフィールドを持ってしまうからである。

この場合は data を使う。

```
data Point = Point Double Double
```

4.2 タプルにまとめれば newtype にできる

ただし、2 つの Double を 1 つのタプルにまとめれば、newtype として定義できる。

```
newtype Point = Point (Double, Double)
```

この場合、フィールドは

```
(Double, Double)
```

という 1 つの型である。

したがって、

```
Point :: (Double, Double) -> Point
```

となり、newtype の制約を満たす。

5 data があるのに newtype がある理由

5.1 型安全性だけなら data でもよい

次のように書けば，data でも UserId と Int は別の型になる．

```
data UserId = UserId Int
```

したがって，型安全性だけなら data でも得られる．

5.2 newtype は実行時コストのないラッパー

では，なぜ newtype があるのか．

理由は，

型検査上は新しい型として扱いたい，実行時には中身の型と同じ軽さで扱いたい

からである．

```
newtype UserId = UserId Int
```

は，型検査上は UserId と Int を区別する．しかし実行時表現としては，基本的に中身の Int と同じように扱われる．

したがって newtype は，

型安全性と実行時効率を両立する仕組み

である．

5.3 type, newtype, data の比較

宣言	新しい型か	実行時の軽さ	用途
type UserId = Int	いいえ	最軽量	読みやすい別名
newtype UserId = UserId Int	はい	最軽量	意味の境界を作る
data UserId = UserId Int	はい	通常データ型	構造・選択肢を作る

6 data の基本

6.1 複数フィールドを持つ型

次の定義を考える.

```
data Point = Point Double Double
deriving Show
```

これは,

Point という型を定義し, その値は Double と Double を持つ Point コンストラクタで作る

という意味である.

値コンストラクタ Point の型は次である.

```
Point :: Double -> Double -> Point
```

値は次のように作る.

```
p :: Point
p = Point 1.0 2.0
```

パターンマッチで分解できる.

```
distanceFromOrigin :: Point -> Double
distanceFromOrigin (Point x y) =
  sqrt (x * x + y * y)
```

ここで,

```
(Point x y)
```

は, Point コンストラクタで作られた値から, 2つの成分を取り出すパターンである.

7 data と直積型・直和型

7.1 Point は直積型

```
data Point = Point Double Double
```

は、典型的な直積型である。構造だけを見ると、

$$\text{Point} \cong \text{Double} \times \text{Double}$$

と見なせる。

すなわち、Point 型の値は

- 1 つ目の Double
- 2 つ目の Double

を同時に持つ。

7.2 複数コンストラクタは直和型

次のような型を考える。

```
data TrafficLight
  = Red
  | Yellow
  | Green
  deriving Show
```

これは、

TrafficLight 型の値は、Red, Yellow, Green のいずれかである

という意味である。

コンストラクタの型は次である。

```
Red    :: TrafficLight
Yellow :: TrafficLight
Green  :: TrafficLight
```

これは直和型である。

$$\text{TrafficLight} \cong 1 + 1 + 1$$

のように考えられる。

7.3 各コンストラクタの中は直積, コンストラクタ同士は直和

より一般的には, 次のような型を考える.

```
data Shape
  = Circle Double
  | Rectangle Double Double
  | PointShape
deriving Show
```

それぞれのコンストラクタの型は次である.

```
Circle      :: Double -> Shape
Rectangle   :: Double -> Double -> Shape
PointShape  :: Shape
```

数学的には,

$$\text{Shape} \cong \text{Double} + (\text{Double} \times \text{Double}) + 1$$

と読める.

すなわち,

- Circle Double は Double を 1 つ持つ場合
- Rectangle Double Double は Double を 2 つ持つ場合
- PointShape は追加情報を持たない場合

である.

このように, Haskell の data は

各コンストラクタの中は直積, コンストラクタ同士は直和

として読める.

このため data で定義される型は ADT, すなわち Algebraic Data Type, 代数的データ型と呼ばれる.

8 | はガードではなくコンストラクタの選択肢

関数定義におけるガードは次のようなものである.

```
abs' :: Int -> Int
abs' x
  | x >= 0    = x
  | otherwise = -x
```

一方, `data` 宣言における `|` は, ガードではなくコンストラクタの選択肢である.

```
data Shape
  = Circle Double
  | Rectangle Double Double
  | PointShape
```

この `|` は,

`Circle Double` または `Rectangle Double Double` または `PointShape`

という直和の区切りである.

9 Maybe と Just

9.1 Maybe の定義

Maybe は概念的には次のような型である。

```
data Maybe a
  = Nothing
  | Just a
```

これは、

Maybe a 型の値は、Nothing であるか、Just a である

という意味である。

コンストラクタの型は次である。

```
Nothing :: Maybe a
Just     :: a -> Maybe a
```

9.2 Just は値を一段持ち上げる

```
Just 5
```

は、5 そのものではない。

```
5      :: Int
Just 5 :: Maybe Int
```

である。

つまり、Just は普通の値を Maybe という文脈に一段持ち上げるコンストラクタである。

```
Just :: a -> Maybe a
```

したがって、

```
Just 5 :: Maybe Int
```

である。

9.3 Maybe Int は Int | Nothing そのものではない

直感的には、Maybe Int は「Int または Nothing」に近い。しかし、より正確には、Haskell では次のようなタグ付き直和型として表される。

```
data Maybe Int
  = Nothing
  | Just Int
```


したがって、

```
Just 5
```

は、

Maybe Int の中の、Just 側に入った 5

である。

数学的には、

$$\text{Maybe } a \cong 1 + a$$

と見られる。

特に、

$$\text{Maybe Int} \cong 1 + \text{Int}$$

である。

ここで、

- Nothing は 1 側、すなわち「値がない」というただ一つのケース
- Just n は Int 側、すなわち「Int の値がある」というケース

である。

9.4 パターンマッチによる取り出し

Maybe の中身を使うには、Nothing の場合と Just の場合を分ける。

```
useMaybe :: Maybe Int -> Int
useMaybe Nothing  = 0
useMaybe (Just n) = n
```

ここで、

```
(Just n)
```

は、Just で包まれた値を取り出すパターンである。

例えば、

```
useMaybe (Just 5)
```

は 5 を返す。

一方、

```
useMaybe Nothing
```

は 0 を返す。

10 実践例：3 次元 bra, ket, operator

10.1 内部表現

3 次元の量子状態を題材に，newtype を使って Bra, Ket, Operator を定義する．

まず複素数，3 次元ベクトル，3x3 行列の内部表現を定義する．

```
import Data.Complex

type C = Complex Double

type Vec3 = (C, C, C)

type Mat3 = (Vec3, Vec3, Vec3)
```

ここで，

```
type Vec3 = (C, C, C)
```

は，3 次元複素ベクトルを表す．

また，

```
type Mat3 = (Vec3, Vec3, Vec3)
```

は，3 本の行ベクトルからなる 3x3 複素行列を表す．

10.2 newtype による意味の境界

次に，newtype で意味を分ける．

```
newtype Ket = Ket
{ unKet :: Vec3
}
deriving (Eq, Show)

newtype Bra = Bra
{ unBra :: Vec3
}
deriving (Eq, Show)

newtype Operator = Operator
{ unOperator :: Mat3
}
deriving (Eq, Show)
```

内部的には Bra と Ket はどちらも Vec3 で表せる．しかし，newtype によって型としては別物になる．

そのため，次のような型シグネチャを書ける．

```
inner :: Bra -> Ket -> C
```

これは数学的な

$$\langle \phi | \psi \rangle$$

に対応する.

10.3 Operator とユニタリ性

```
newtype Operator = Operator Mat3
```

と書くだけでは, 内部の `Mat3` がユニタリ行列であることは保証されない.

したがって, 実践的には `Operator` の値コンストラクタをモジュール外には公開せず, スマートコンストラクタを用意するのがよい.

```
mkOperator :: Mat3 -> Maybe Operator
mkOperator m
  | isUnitary 1.0e-10 m = Just (Operator m)
  | otherwise           = Nothing
```

このようにすると,

`Operator` 型の値は, 生成時にユニタリ性検査を通過したもの

という不変条件を作れる.

ここで `Maybe Operator` を返すのは, 入力行列がユニタリとは限らないからである.

```
Just op
```

ならユニタリ性検査に成功したことを表し,

```
Nothing
```

なら失敗したことを表す.

10.4 `Mat3` を `Matrix` ではなく `(Vec3, Vec3, Vec3)` にした理由

今回,

```
type Mat3 = (Vec3, Vec3, Vec3)
```

とした主な理由は次である.

1. 外部ライブラリに依存しない.
2. `3x3` であることが型の形に直接現れる.
3. `newtype` の 1 フィールド制約と相性がよい.
4. パターンマッチで行列演算を明示的に書ける.

5. newtype の復習教材として分かりやすい.

一般的な Matrix C という型を使う場合, それが 3x3 であるかどうかは, 多くの場合, 型からは分からず, 実行時にチェックする必要がある.

一方,

```
type Vec3 = (C, C, C)
type Mat3 = (Vec3, Vec3, Vec3)
```

であれば, 構造上 3 行 3 列であることが明示される.

ただし, 実用的な数値計算では, 専用の行列ライブラリを使う方が適切な場合も多い. 特に, 固有値計算, 特異値分解, BLAS/LAPACK の利用, 大規模行列演算などを行う場合は, 外部ライブラリの利用を検討すべきである.

11 deriving のまとめ

11.1 deriving とは何か

deriving は、型クラスのインスタンスを Haskell コンパイラに自動生成させる仕組みである。
例えば次のような型を考える。

```
data Color
= Red
| Green
| Blue
deriving (Eq, Ord, Show)
```

この deriving (Eq, Ord, Show) により、Color 型に対して次のような操作が自動的に使えるようになる。

- Eq: 等値比較
- Ord: 順序比較
- Show: 文字列表示

つまり、deriving は

この型は、この型クラスの振る舞いを標準的な方法で持てるので、その実装を自動生成してほしい

という宣言である。

11.2 最頻出の 3 つ : Eq, Ord, Show

11.2.1 Eq

Eq は、等しいかどうかを比較できることを表す型クラスである。

代表的な演算子は次である。

```
(==) :: Eq a => a -> a -> Bool  
(/=) :: Eq a => a -> a -> Bool
```

例：

```
data Color  
  = Red  
  | Green  
  | Blue  
  deriving (Eq, Show)  
  
test1 :: Bool  
test1 = Red == Red  
  
test2 :: Bool  
test2 = Red == Blue
```

Eq は「比較」と言っても、正確には「等値比較」である。

11.2.2 Ord

Ord は、順序比較ができることを表す型クラスである。

代表的な関数・演算子は次である。

```
compare :: Ord a => a -> a -> Ordering
```

```
(<) :: Ord a => a -> a -> Bool
```

```
(<=) :: Ord a => a -> a -> Bool
```

```
(>) :: Ord a => a -> a -> Bool
```

```
(>=) :: Ord a => a -> a -> Bool
```

例：

```
data Grade
```

```
  = C
```

```
  | B
```

```
  | A
```

```
  deriving (Eq, Ord, Show)
```

この場合、定義順に従って、

```
C < B
```

```
B < A
```

```
A > C
```

が成り立つ。

したがって、Ord は「順序集合に属することが可能」という理解に近い。ただし Haskell 的には、

その型に順序比較操作を与える型クラス

と理解するとよい。

また、Ord は Eq を前提にする。

概念的には次のような関係である。

```
class Eq a => Ord a where
```

```
  compare :: a -> a -> Ordering
```

つまり、順序比較できるなら等値比較もできる、という設計である。

11.2.3 Show

Show は、値を文字列として表現できることを表す型クラスである。

代表的な関数は次である。

```
show :: Show a => a -> String
```

また、

```
print :: Show a => a -> IO ()
```

も Show に依存している.

概念的には,

```
print x = putStrLn (show x)
```

に近い.

例:

```
data Color
  = Red
  | Green
  | Blue
deriving Show
```

このとき,

```
show Red
```

は,

```
"Red"
```

になる.

Show は, 主にデバッグ表示, GHCi での確認, ログ出力に便利である. ただし, ユーザー向けのきれいな表示や数式表示をしたい場合は, 自動導出された Show ではなく, 自分で表示関数を定義する方がよいことも多い.

11.3 列挙型でよく使う Enum と Bounded

列挙型では、Enum と Bounded が便利である。

```
data Direction
  = North
  | East
  | South
  | West
deriving (Eq, Ord, Show, Enum, Bounded)
```

11.3.1 Enum

Enum は、値を列挙できることを表す型クラスである。

代表的には次が使える。

```
succ :: Enum a => a -> a
pred :: Enum a => a -> a
```

例：

```
succ North
-- East

pred South
-- East
```

また、範囲構文も使える。

```
[North .. West]
-- [North, East, South, West]
```

11.3.2 Bounded

Bounded は、その型に最小値と最大値があることを表す型クラスである。

代表的な値は次である。

```
minBound :: Bounded a => a
maxBound :: Bounded a => a
```

例えば Direction では、

```
minBound :: Direction
-- North

maxBound :: Direction
-- West
```

となる。

Enum と Bounded を組み合わせると、その型の全要素を列挙できる。

```
allDirections :: [Direction]
allDirections = [minBound .. maxBound]
```

この結果は、

```
[North, East, South, West]
```

である。

11.4 Read

Read は、文字列から値を読むための型クラスである。Show の逆向きに近いものと考えられる。

```
data Color
  = Red
  | Green
  | Blue
  deriving (Show, Read)
```

このとき、

```
read "Red" :: Color
```

が使える。

ただし、read は失敗すると実行時エラーになる。

```
read "Purple" :: Color
```

は危険である。

実用的には、Text.Read の readMaybe を使う方が安全である。

```
import Text.Read

readMaybe "Red" :: Maybe Color
readMaybe "Purple" :: Maybe Color
```

11.5 Ix

Ix は、値を配列の添字範囲として使えることを表す型クラスである。

```
import Data.Ix

data Day
  = Mon | Tue | Wed | Thu | Fri
  deriving (Eq, Ord, Enum, Bounded, Ix, Show)
```

例えば、

```
range (Mon, Fri)
```

によって,

```
[Mon, Tue, Wed, Thu, Fri]
```

のような範囲を得られる.

ただし, 通常の入門段階では `Ix` は深追いしなくてよい. 配列 `Array` を使う段階で意識すれば十分である.

11.6 GHC 拡張で導出できるもの

GHC では, 言語拡張を使うことで, より多くの型クラスを `deriving` できる.

代表例は次である.

```
{-# LANGUAGE DeriveFunctor #-}  
{-# LANGUAGE DeriveFoldable #-}  
{-# LANGUAGE DeriveTraversable #-}  
{-# LANGUAGE DeriveGeneric #-}
```

11.6.1 Functor

`Functor` は, 型引数を 1 つ持つ構造の中身に関数を適用できることを表す.

```
{-# LANGUAGE DeriveFunctor #-}  
  
data Box a = Box a  
  deriving (Show, Functor)
```

このとき,

```
fmap (+1) (Box 10)
```

は,

```
Box 11
```

になる.

11.6.2 Foldable

`Foldable` は, 中身を畳み込める構造であることを表す.

```
{-# LANGUAGE DeriveFoldable #-}  
  
data Box a = Box a  
  deriving (Show, Foldable)
```

例えば,

```
sum (Box 10)
```

のような操作ができる.

11.6.3 Traversable

Traversable は、効果付きで中身を走査できる構造であることを表す。

```
{-# LANGUAGE DeriveTraversable #-}

data Box a = Box a
  deriving (Show, Functor, Foldable, Traversable)
```

Functor, Foldable, Traversable は、型引数を持つデータ構造を一般的に扱うときに重要になる。

11.6.4 Generic

Generic は、汎用プログラミングのための型クラスである。

```
{-# LANGUAGE DeriveGeneric #-}

import GHC.Generics

data Person = Person
  { name :: String
  , age  :: Int
  }
  deriving (Show, Generic)
```

Generic は、JSON 変換、設定ファイル、シリアライズ、テストデータ生成などのライブラリで内部的によく使われる。

11.7 newtype と GeneralizedNewtypeDeriving

newtype では、内部表現の型が持っているインスタンスを流用したくなることが多い。

例えば、

```
newtype Score = Score Int
```

に対して、Int と同じように足し算をしたい場合がある。

GHC 拡張 GeneralizedNewtypeDeriving を使うと、次のように書ける。

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

newtype Score = Score Int
  deriving (Eq, Ord, Show, Num)
```

このとき、

```
Score 10 + Score 20
```

が可能になる。

ただし、意味的に自然な場合に限って使うべきである。

例えば、

```
newtype UserId = UserId Int
deriving (Eq, Ord, Show, Num)
```

は危険である.

なぜなら,

```
UserId 10 + UserId 20
```

ができてしまうからである.

ユーザー ID に足し算が自然でないなら, Num を導出すべきではない.

11.8 DerivingStrategies

GHC では, どの方式で deriving するかを明示することもできる.

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

newtype Score = Score Int
deriving stock (Eq, Show)
deriving newtype (Ord, Num)
```

ここで,

- deriving stock は通常の標準的な導出
- deriving newtype は内部表現のインスタンスを流用する導出

を意味する.

コードが大きくなると, どの導出方式を使っているのかを明示することは可読性の面で有効である.

11.9 Ket, Bra, Operator で何を derive すべきか

今回の量子ベクトルの例では、まずは次で十分である。

```
newtype Ket = Ket
{ unKet :: Vec3
}
deriving (Eq, Show)

newtype Bra = Bra
{ unBra :: Vec3
}
deriving (Eq, Show)

newtype Operator = Operator
{ unOperator :: Mat3
}
deriving (Eq, Show)
```

Eq は、内部表現が等しいかどうかを比較するために使える。Show は、GHCi やデバッグで値を確認するために使える。

一方、Ord は通常は自然ではない。なぜなら、複素数やベクトルや行列には、標準的で自然な全順序がないからである。

また、Num を導出するかどうかにも慎重に考えるべきである。Ket + Ket や Operator * Operator のような演算をどう定義するかは、数学的意味に合わせて設計すべきである。

11.10 まとめ

deriving は、型クラスインスタンスの標準的な実装を自動生成する仕組みである。最初に押さえるべきものは次である。

型クラス	意味	代表的な用途
Eq	等値比較できる	==, /=
Ord	順序比較できる	<, <=, compare
Show	文字列化できる	show, print, GHCi 表示
Read	文字列から読める	read, readMaybe
Enum	列挙できる	succ, pred, [A .. B]
Bounded	最小値・最大値を持つ	minBound, maxBound
Ix	添字範囲として使える	range, Array

学習段階では、まず

```
deriving (Eq, Show)
```

を基本とし、列挙型では

```
deriving (Eq, Ord, Show, Enum, Bounded)
```

をよく使う、と理解するとよい。

より実用的なコードでは、型引数を持つデータ構造に対して

```
deriving (Functor, Foldable, Traversable)
```

を使ったり、ライブラリ連携のために

```
deriving Generic
```

を使ったりする。

`newtype` では、`GeneralizedNewtypeDeriving` によって内部型のインスタンスを流用できるが、それがドメイン上自然かどうかを常に確認する必要がある。

12 newtype と data における演算定義の違い

12.1 問題意識

type, newtype, data の違いは、演算をどう扱うかにも現れる。
特に重要なのは次の点である。

newtype は内部型の演算を再利用しやすいが、data は必要に応じて演算の意味を明示的に定義する必要がある。

ただし、これは

newtype なら何もしなくてもすべての演算が使える

という意味ではない。

正確には、

- type は alias なので、元の型の演算をそのまま使える。
- newtype は新しい型だが、内部型と同じ実行時表現を持つため、GHC 拡張により内部型のインスタンスを流用しやすい。
- data は通常の代数的データ型なので、演算の意味を自分で定義する場面が多い。

12.2 type の場合：元の型そのもの

まず type の場合を考える。

```
type Score = Int
```

これは単なる alias なので、Score は型検査上も Int と同じである。

したがって、Int の演算はそのまま使える。

```
x :: Score
x = 10

y :: Score
y = 20

z :: Score
z = x + y
```

ここで (+) は Int の加算そのものである。Score という名前は読みやすさを与えるが、型安全性は増えない。

12.3 newtype の場合：新しい型だが演算を流用しやすい

次に newtype を考える.

```
newtype Score = Score Int
deriving Show
```

この場合、Score は Int とは別の型である. そのため、このままでは次のような加算はできない.

```
Score 10 + Score 20
```

なぜなら、Score が Num のインスタンスではないからである.

しかし、newtype は内部型と同じ実行時表現を持つため、GHC 拡張を使うと内部型の型クラスインスタンスを流用できる.

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

newtype Score = Score Int
deriving (Eq, Ord, Show, Num)
```

このようにすると,

```
Score 10 + Score 20
```

が可能になる.

これは内部の Int が持つ Num インスタンスを、Score に対して流用していると考えればよい.

12.4 deriving newtype による明示

DerivingStrategies を使うと、どの方式で導出しているのかを明示できる.

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

newtype Score = Score Int
deriving stock (Eq, Show)
deriving newtype (Ord, Num)
```

ここで,

- deriving stock は標準的な導出
- deriving newtype は内部型のインスタンスを流用する導出

である.

この書き方は、newtype を設計上の意味境界として使いつつ、内部表現の演算を再利用したい場合に有効である.

12.5 data の場合：演算の意味を明示的に定義する

一方, data で同じような型を定義するとする.

```
data Score = Score Int
  deriving (Eq, Ord, Show)
```

この場合, Eq, Ord, Show は標準的に導出できる.

例えば,

```
Score 10 == Score 10
Score 10 < Score 20
show (Score 10)
```

は使える.

しかし, Num は通常, このように簡単には導出できない.

```
data Score = Score Int
  deriving (Eq, Ord, Show, Num)
```

のようには普通は書けない.

そのため, Score に足し算を定義したいなら, 次のように自分で Num インスタンスを書く必要がある.

```
data Score = Score Int
  deriving (Eq, Ord, Show)

instance Num Score where
  Score x + Score y = Score (x + y)
  Score x * Score y = Score (x * y)
  negate (Score x) = Score (negate x)
  abs (Score x) = Score (abs x)
  signum (Score x) = Score (signum x)
  fromInteger n = Score (fromInteger n)
```

このように, data の場合は,

この型にとって足し算とは何か, 掛け算とは何か

を明示的に定義する必要がある.

12.6 なぜ newtype では流用しやすいのか

newtype は, 型検査上は新しい型を作るが, 実行時表現としては内部型と同じである.

```
newtype Score = Score Int
```

は, 型としては Score と Int を区別する. しかし実行時には, Score は基本的に Int と同じ表現で扱われる.

そのため、GHC は内部型のインスタンスを `newtype` 側に安全に持ち上げやすい。
一方、

```
data Score = Score Int
```

は通常の代数的データ型である。 `newtype` のように「内部型と同じ表現である」とまでは言えない。
したがって、 `Int` の `Num` インスタンスをそのまま `Score` に流用する、という扱いにはならない。

12.7 ただし、 `newtype` でも意味的な妥当性が重要

`newtype` で内部型の演算を流用できるからといって、何でも導出してよいわけではない。
例えば、次のような定義は危険である。

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}  
  
newtype UserId = UserId Int  
    deriving (Eq, Ord, Show, Num)
```

この場合、

```
UserId 10 + UserId 20
```

ができてしまう。

しかし、ユーザー ID に足し算が定義されるのは、通常は不自然である。
したがって、 `newtype` で演算を流用するときには、

その演算がドメイン上、意味を持つかどうか

を確認する必要がある。

12.8 量子ベクトルの例で考える

今回の量子ベクトルの例では、次のような型を定義した。

```
newtype Ket = Ket  
    { unKet :: Vec3  
    }  
    deriving (Eq, Show)  
  
newtype Bra = Bra  
    { unBra :: Vec3  
    }  
    deriving (Eq, Show)  
  
newtype Operator = Operator  
    { unOperator :: Mat3  
    }  
    deriving (Eq, Show)
```

ここでは Eq と Show の導出は自然である.

- Eq は内部表現が等しいかどうかを調べるため
- Show は GHCi やデバッグで表示するため

一方, Num を単純に導出するかどうかは慎重に考えるべきである.

例えば, 数学的には ket ベクトル同士の和は定義できる.

$$|\psi\rangle + |\phi\rangle$$

また, スカラー倍も定義できる.

$$c|\psi\rangle$$

しかし, Haskell の Num は足し算だけでなく, 掛け算, fromInteger, abs, signum などとも要求する. したがって, 単に

```
deriving newtype Num
```

とするより, 量子ベクトルに必要な演算を個別に設計する方が自然な場合も多い.

例えば,

```
addKet :: Ket -> Ket -> Ket
scaleKet :: C -> Ket -> Ket
applyOperator :: Operator -> Ket -> Ket
inner :: Bra -> Ket -> C
```

のように, 数式上の意味に合わせて関数を定義する方が明確である.

12.9 比較表

構文	型としての扱い	演算の扱い	注意点
<code>type Score = Int</code>	Int と同じ	Int の演算をそのまま使える	型安全性は増えない
<code>newtype Score = Score Int</code>	Int とは別型	内部型の演算を流用しやすい	意味的に自然な演算だけ
<code>data Score = Score Int</code>	新しい代数的データ型	必要に応じて演算を定義する	演算の意味を明示する必

12.10 まとめ

演算定義の観点では, 次のように整理できる.

- type は alias なので, 元の型の演算をそのまま使える.
- newtype は新しい型だが, 内部型の演算を流用しやすい.
- data は本格的なデータ型なので, 演算の意味を必要に応じて自分で定義する.

したがって, newtype は

意味の境界を作りつつ、内部表現の演算を必要に応じて再利用できる

という点で非常に強力である.

ただし、内部型の演算を流用できることと、その演算がドメイン上妥当であることは別である.
Haskell の型設計では、単に導出できるかではなく、

その演算をこの型に許してよいか

を考えることが重要である.

13 この後半で扱う流れ

このノートの後半では、次の流れで Haskell の抽象化を整理する.

Just 5 :: Maybe Int の「一段持ち上げる」感覚

↓

Maybe / Either

↓

Functor

↓

Applicative

↓

Monad

ここでの主眼は、Monad をいきなり特別なものとして扱うのではなく、Maybe や Either のような「文脈付きの値」を、どのように変換し、組み合わせ、次の計算へ接続するかという観点から理解することである.

14 Maybe / Either から Functor, Applicative, Monad へ

14.1 出発点：Just 5 :: Maybe Int は一段持ち上がっている

ここまでで、次のことを確認した。

```
5      :: Int
Just 5 :: Maybe Int
```

つまり、Just 5 は 5 そのものではない。5 :: Int を、Maybe という文脈に一段持ち上げた値である。

```
Just :: a -> Maybe a
```

したがって、Just は

通常の値を、Maybe という計算文脈に入れるコンストラクタ

である。

ここで重要なのは、いったん値が

```
Maybe Int
```

の中に入ると、通常の

```
Int -> Int
```

の関数をそのまま適用できない、という点である。

例えば、

```
double :: Int -> Int
double x = x * 2
```

があるとする。

これは、

```
double 5
```

には使える。しかし、

```
double (Just 5)
```

は使えない。

なぜなら、

```
double :: Int -> Int
Just 5  :: Maybe Int
```

であり、型が合わないからである。

ここから自然に出てくる問いは次である。

普通の関数 $\text{Int} \rightarrow \text{Int}$ を、 Maybe Int の中身に適用するにはどうすればよいか。

この問いへの最初の答えが `Functor` である。

14.2 Maybe の直感：失敗するかもしれない値

`Maybe` は、値が存在しない可能性を型で表す。

```
data Maybe a
  = Nothing
  | Just a
```

これは数学的には、

$$\text{Maybe } a \cong 1 + a$$

と見ることができる。

- `Nothing` は「値がない」という 1 個のケース
- `Just a` は「値がある」というケース

例えば、安全な割り算は次のように書ける。

```
safeDiv :: Double -> Double -> Maybe Double
safeDiv _ 0 = Nothing
safeDiv x y = Just (x / y)
```

この関数は、通常の割り算と違って、失敗する可能性を型に明示している。

```
safeDiv 10 2
-- Just 5.0

safeDiv 10 0
-- Nothing
```

したがって、

```
Maybe Double
```

は、

成功して `Double` が得られるかもしれないし、失敗して値がないかもしれない

という意味を持つ。

14.3 Either の直感：失敗理由を持つ値

`Maybe` は、失敗したことは分かるが、なぜ失敗したのかは表さない。

失敗理由も持ちたいときには `Either` を使う。

概念的には次のような型である。


```
data Either e a
= Left e
| Right a
```

よくある使い方では,

- `Left e` は失敗とその理由
- `Right a` は成功値

を表す.

例えば,

```
safeDivE :: Double -> Double -> Either String Double
safeDivE _ 0 = Left "division by zero"
safeDivE x y = Right (x / y)
```

と書ける.

この場合,

```
safeDivE 10 2
-- Right 5.0

safeDivE 10 0
-- Left "division by zero"
```

である.

`Either e a` は数学的には,

$$\text{Either } e \ a \cong e + a$$

と見ることができる.

`Maybe a` が

$$1 + a$$

であったのに対して, `Either e a` は

$$e + a$$

である.

つまり, `Nothing` という単一の失敗ではなく, `e` 型の情報を持った失敗を表せる.

14.4 Maybe と Either の関係

`Maybe` は, 失敗理由を持たない単純な失敗である.

```
Maybe a
```

は,

値があるか, ないか

を表す.

一方,

`Either e a`

は,

失敗理由 `e` があるか, 成功値 `a` があるか

を表す.

したがって, おおまかには,

`Maybe a`

は

`Either () a`

に近いと考えられる.

ここで `()` は unit 型であり, 値を 1 つだけ持つ型である.

`() :: ()`

したがって,

$$\text{Maybe } a \cong 1 + a$$

であり,

$$\text{Either } () a \cong 1 + a$$

である.

この意味で, `Maybe` は `Either` の失敗理由を単位型にしたものと見ることができる.

15 Functor：中身に関数を適用する

15.1 問題：Maybe Int の中の Int に関数を適用したい

もう一度，次の関数を考える．

```
double :: Int -> Int
double x = x * 2
```

これは，

```
5 :: Int
```

には使えるが，

```
Just 5 :: Maybe Int
```

にはそのまま使えない．

そこで欲しいのは，次のような働きをする関数である．

```
Maybe Int -> Maybe Int
```

より一般には，

```
(a -> b) -> Maybe a -> Maybe b
```

である．

これを与えるのが fmap である．

15.2 fmap の型

Functor 型クラスの代表的な関数は fmap である.

```
fmap :: Functor f => (a -> b) -> f a -> f b
```

これは,

普通の関数 $a \rightarrow b$ を, 文脈付きの値 $f\ a$ の中身に適用して, $f\ b$ を得る

という意味である.

Maybe に特殊化すると,

```
fmap :: (a -> b) -> Maybe a -> Maybe b
```

である.

具体例:

```
fmap double (Just 5)
-- Just 10

fmap double Nothing
-- Nothing
```

15.3 Maybe に対する fmap の動き

Maybe に対する fmap は, 概念的には次のように定義できる.

```
fmapMaybe :: (a -> b) -> Maybe a -> Maybe b
fmapMaybe _ Nothing = Nothing
fmapMaybe f (Just x) = Just (f x)
```

つまり,

- Nothing なら何もできないので Nothing のまま
- Just x なら中身の x に関数 f を適用し, 結果を再び Just で包む

である.

この「再び包む」が重要である. fmap は, Maybe という文脈を壊さず, 中身だけを変換する.

15.4 演算子 <\$>

fmap には中置演算子版がある.

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
```

実質的には fmap と同じである.

したがって,

```
fmap double (Just 5)
```

は,

```
double <$> Just 5
```

と書ける.

結果はどちらも,

```
Just 10
```

である.

15.5 Either に対する fmap

Either e も Functor である.

ただし, Either は型引数を 2 つ持つ.

```
Either e a
```

Functor は型引数を 1 つ取る型コンストラクタに対する型クラスである. そのため, Either e のように, 左側の型引数 e を固定して考える.

```
fmap :: (a -> b) -> Either e a -> Either e b
```

動きは次のようになる.

```
fmap (+1) (Right 5)
-- Right 6

fmap (+1) (Left "error")
-- Left "error"
```

つまり, Right 側の成功値にだけ関数を適用し, Left 側のエラーはそのまま保持する.

概念的には次のように書ける.

```
fmapEither :: (a -> b) -> Either e a -> Either e b
fmapEither _ (Left e)  = Left e
fmapEither f (Right x) = Right (f x)
```

15.6 Functor の直感

Functor は,

何らかの文脈 f の中にある値に, 普通の関数を適用できる構造

である.

Maybe の場合, 文脈は「失敗するかもしれない」である.

```
Maybe a
```

`Either e` の場合、文脈は「失敗理由 `e` を持って失敗するかもしれない」である。

```
Either e a
```

`fmap` は、その文脈を保ったまま、中身だけを変換する。

```
fmap :: (a -> b) -> f a -> f b
```

この段階では、`fmap` が扱えるのは

通常関数 `a -> b` を、文脈付きの値 `f a` に適用する

という形である。

次の段階では、文脈付き関数を文脈付きの値に適用したくなる。これが `Applicative` である。

16 Applicative：文脈付きの関数を文脈付きの値に適用する

16.1 Functor だけでは足りない場合

Functor では、次のようなことができた。

```
fmap :: (a -> b) -> f a -> f b
```

つまり、普通の関数を文脈付きの値に適用できる。

しかし、次のような関数を考える。

```
add :: Int -> Int -> Int
add x y = x + y
```

これに対して、

```
Just 3 :: Maybe Int
Just 4 :: Maybe Int
```

を使って、

```
Just 7
```

を得たい。

fmap を使うと、まず次のようになる。

```
fmap add (Just 3)
```

add :: Int -> Int -> Int なので、これは

```
Just (add 3) :: Maybe (Int -> Int)
```

になる。

つまり、関数そのものがMaybeの中に入ってしまう。

ここで欲しいのは、

```
Maybe (Int -> Int) -> Maybe Int -> Maybe Int
```

という形の適用である。

これを与えるのがApplicativeの<*>である。

16.2 Applicative の基本関数

Applicativeには、主に次の2つがある。

```
pure :: Applicative f => a -> f a
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
```

pureは、普通の値を文脈に持ち上げる。

```
pure 5 :: Maybe Int
-- Just 5
```

<*> は、文脈付きの関数を、文脈付きの値に適用する。

```
Just (+1) <*> Just 5
-- Just 6
```

16.3 Maybe に対する Applicative

Maybe の Applicative は、失敗があれば全体が失敗する、という動きをする。

```
Just (+1) <*> Just 5
-- Just 6

Nothing <*> Just 5
-- Nothing

Just (+1) <*> Nothing
-- Nothing
```

つまり、関数側または値側が Nothing なら、結果も Nothing である。

16.4 二引数関数への適用

Applicative を使うと、二引数関数を文脈付きの値に自然に適用できる。

```
add :: Int -> Int -> Int
add x y = x + y
```

に対して、

```
add <$> Just 3 <*> Just 4
-- Just 7
```

と書ける。

これは左から見ると、次のように進む。

```
add <$> Just 3
-- Just (add 3)

Just (add 3) <*> Just 4
-- Just 7
```

ここで、add 3 は

```
Int -> Int
```

型の関数である。

つまり、Haskell の関数はカーリー化されているため、


```
add :: Int -> Int -> Int
```

は,

```
add :: Int -> (Int -> Int)
```

と読める.

このため,

```
add <$> Just 3 <*> Just 4
```

という書き方が成立する.

16.5 pure による持ち上げ

Applicative では, pure によって普通の値を文脈に入れられる.

```
pure 5 :: Maybe Int
-- Just 5
```

したがって, 普通の関数を文脈付き計算として使いたい場合には, 次のようにも書ける.

```
pure add <*> Just 3 <*> Just 4
-- Just 7
```

これは,

```
add <$> Just 3 <*> Just 4
```

とほぼ同じ意味である.

16.6 Either に対する Applicative

Either e の Applicative では, Right が成功, Left が失敗として扱われる.

```
Right (+1) <*> Right 5
-- Right 6

Left "bad function" <*> Right 5
-- Left "bad function"

Right (+1) <*> Left "bad value"
-- Left "bad value"
```

二引数関数でも同じである.

```
add <$> Right 3 <*> Right 4
-- Right 7

add <$> Right 3 <*> Left "second argument failed"
-- Left "second argument failed"
```

16.7 Functor と Applicative の違い

Functor は,

```
fmap :: (a -> b) -> f a -> f b
```

である.

つまり, 普通の関数を文脈付きの値に適用する.

一方, Applicative は,

```
(<*>) :: f (a -> b) -> f a -> f b
```

である.

つまり, 文脈付きの関数を, 文脈付きの値に適用する.

表にすると次のようになる.

型クラス	代表的な型	意味
Functor	$(a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	普通の関数を中身に適用する
Applicative	$f\ (a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	文脈付き関数を文脈付き値に適用する

ここまでで, 複数の文脈付き値を使って, 通常関数を適用できるようになった.

しかし, まだ問題がある.

次の段階では,

前の計算結果によって, 次にどの計算をするかが変わる

場合を考える.

これが Monad である.

17 Monad：結果に応じて次の計算を選ぶ

17.1 Applicative では表しにくい場合

Applicative は、独立した文脈付き値を組み合わせるのに向いている。

例えば、

```
add <$> Just 3 <*> Just 4
```

では、Just 3 と Just 4 は互いに独立である。

しかし、次のような計算を考える。

まず割り算をする。その結果が得られたら、さらにその逆数を取る。

例えば、

```
safeDiv :: Double -> Double -> Maybe Double
safeDiv _ 0 = Nothing
safeDiv x y = Just (x / y)

safeRecip :: Double -> Maybe Double
safeRecip 0 = Nothing
safeRecip x = Just (1 / x)
```

ここで、

```
safeDiv 10 2
```

の結果が Just 5.0 なら、その中身 5.0 に対して safeRecip を実行したい。

しかし、safeRecip の型は

```
safeRecip :: Double -> Maybe Double
```

である。

つまり、普通の値を受け取って、再び Maybe に入った値を返す。

このような関数をつなぐためには、

```
Maybe a -> (a -> Maybe b) -> Maybe b
```

という形が必要になる。

これが Monad の >=> である。

17.2 >=> の型

Monad の代表的な演算子は >=> である。これは bind と呼ばれる。

```
(>=>) :: Monad m => m a -> (a -> m b) -> m b
```

読み方は次のように考えるとよい。

文脈付きの値 $m\ a$ から、中身 a を取り出せる場合には、次の計算 $a \rightarrow m\ b$ に渡し、結果として $m\ b$ を得る。

Maybe に特殊化すると、

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

である。

17.3 Maybe に対する bind

Maybe に対する bind は、概念的には次のように定義できる。

```
bindMaybe :: Maybe a -> (a -> Maybe b) -> Maybe b
bindMaybe Nothing _ = Nothing
bindMaybe (Just x) f = f x
```

つまり、

- Nothing なら、次の計算は実行できないので Nothing
- Just x なら、中身 x を次の関数 f に渡す

である。

ここで重要なのは、 $f\ x$ はすでに Maybe b を返すため、もう一度 Just で包まないという点である。

17.4 fmap と >>= の違い

fmap は次の型だった。

```
fmap :: (a -> b) -> Maybe a -> Maybe b
```

つまり、中身に適用する関数は

```
a -> b
```

である。

一方、bind は次の型である。

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

つまり、中身に適用する関数は

```
a -> Maybe b
```

である。

この違いが非常に重要である。

操作	関数の型	意味
fmap	$a \rightarrow b$	普通関数を中身に適用する
>>=	$a \rightarrow m\ b$	次の文脈付き計算へ進む

17.5 例：安全な計算をつなぐ

次の関数を使う.

```
safeDiv :: Double -> Double -> Maybe Double
safeDiv _ 0 = Nothing
safeDiv x y = Just (x / y)

safeRecip :: Double -> Maybe Double
safeRecip 0 = Nothing
safeRecip x = Just (1 / x)
```

これらを bind でつなぐ.

```
example1 :: Maybe Double
example1 =
    safeDiv 10 2 >=> safeRecip
```

計算は次のように進む.

```
safeDiv 10 2
-- Just 5.0

Just 5.0 >=> safeRecip
-- safeRecip 5.0
-- Just 0.2
```

したがって,

```
example1
-- Just 0.2
```

である.

一方,

```
example2 :: Maybe Double
example2 =
    safeDiv 10 0 >=> safeRecip
```

では,

```
safeDiv 10 0
-- Nothing

Nothing >=> safeRecip
-- Nothing
```

となる.

つまり, 途中で失敗すると, 後続の計算は実行されず, 全体が失敗する.

17.6 do 記法

`bind` をたくさん使うと読みにくくなる.

例えば,

```
safeCalc :: Double -> Double -> Maybe Double
safeCalc x y =
  safeDiv x y >=> safeRecip
```

程度ならよいが, 複数段になると読みにくい.

そこで `do` 記法を使う.

```
safeCalc :: Double -> Double -> Maybe Double
safeCalc x y = do
  q <- safeDiv x y
  r <- safeRecip q
  pure r
```

これは, 概念的には次の `bind` の連鎖である.

```
safeCalc x y =
  safeDiv x y >=> \q ->
  safeRecip q >=> \r ->
  pure r
```

ここで,

```
q <- safeDiv x y
```

は,

`safeDiv x y` が成功して `Just q` になったら, その中身を `q` として次へ進む. 失敗して `Nothing` なら, 全体も `Nothing` になる

と読める.

17.7 do 記法と命令型プログラムの見た目

`do` 記法は, 見た目としては命令型プログラムに似ている.

```
safeCalc x y = do
  q <- safeDiv x y
  r <- safeRecip q
  pure r
```

しかし, 意味としては, これは代入ではない. `q <- safeDiv x y` は,

`Maybe` 文脈の計算を行い, 成功したときだけ中身を `q` として次に渡す

という bind の構文糖衣である.

つまり, do 記法は,

文脈付き計算を順に接続する記法

である.

17.8 Either に対する Monad

Either e も Monad である.

Either e に対する bind は, 概念的には次のように動く.

```
bindEither :: Either e a -> (a -> Either e b) -> Either e b
bindEither (Left e) _ = Left e
bindEither (Right x) f = f x
```

つまり,

- Left e なら, エラーを保持してそのまま終了
- Right x なら, 中身 x を次の計算に渡す

である.

例:

```
safeDivE :: Double -> Double -> Either String Double
safeDivE _ 0 = Left "division by zero"
safeDivE x y = Right (x / y)

safeRecipE :: Double -> Either String Double
safeRecipE 0 = Left "reciprocal of zero"
safeRecipE x = Right (1 / x)

safeCalcE :: Double -> Double -> Either String Double
safeCalcE x y = do
  q <- safeDivE x y
  r <- safeRecipE q
  pure r
```

この場合,

```
safeCalcE 10 2
-- Right 0.2
```

である.

一方,

```
safeCalcE 10 0
-- Left "division by zero"
```

である。

Either では、失敗時に理由が残るため、Maybe よりもデバッグしやすいことが多い。

18 Functor, Applicative, Monad の関係

18.1 型クラスの包含関係

現在の Haskell では、概念的に次の関係がある。

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  (>=) :: m a -> (a -> m b) -> m b
```

つまり、

$$\text{Monad} \Rightarrow \text{Applicative} \Rightarrow \text{Functor}$$

である。

言い換えると、Monad であるものは、Applicative でもあり、Functor でもある。

18.2 段階的な理解

ここまでの流れを、計算能力の増加として見ると分かりやすい。

段階	代表的な型	できること
Functor	$(a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	文脈の中身を変換する
Applicative	$f\ (a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	文脈付きの複数の値を関数に渡す
Monad	$m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$	前の結果に応じて次の文脈付き計算を選ぶ

18.3 具体例で比較する

通常の数値：

```
x :: Int
x = 5
```

一段持ち上がった値：

```
mx :: Maybe Int
mx = Just 5
```

普通の関数：

```
double :: Int -> Int
double x = x * 2
```

文脈付きの値に普通の関数を適用する：

```
double <$> mx
-- Just 10
```

二引数関数を、文脈付きの 2 つの値に適用する：

```
add :: Int -> Int -> Int
add x y = x + y

add <$> Just 3 <*> Just 4
-- Just 7
```

前の計算結果に応じて、次の計算を決める：

```
safeDiv 10 2 >=> safeRecip
-- Just 0.2
```

このように、Functor, Applicative, Monad は、まったく別々の概念というより、

文脈付きの値を扱うための、段階的に強いインターフェイス

として理解できる。

18.4 Maybe で見る 3 段階

Maybe を題材に、3 つの段階を整理する。

18.4.1 Functor

```
fmap (+1) (Just 5)
-- Just 6

fmap (+1) Nothing
-- Nothing
```

中身があれば変換し、なければ何もしない。

18.4.2 Applicative

```
(+) <$> Just 3 <*> Just 4
-- Just 7

(+) <$> Just 3 <*> Nothing
-- Nothing
```

複数の Maybe 値を組み合わせる。どれかが Nothing なら全体も Nothing になる。

18.4.3 Monad

```
Just 5 >=> safeRecip
-- Just 0.2

Just 0 >=> safeRecip
-- Nothing

Nothing >=> safeRecip
-- Nothing
```

前の結果が成功したときだけ、次の計算へ進む。

18.5 Either で見る 3 段階

Either String でも同様である。

18.5.1 Functor

```
fmap (+1) (Right 5)
-- Right 6

fmap (+1) (Left "error")
-- Left "error"
```

成功値だけ変換する。

18.5.2 Applicative

```
(+) <$> Right 3 <*> Right 4
-- Right 7

(+) <$> Right 3 <*> Left "bad second"
-- Left "bad second"
```

複数の成功値を組み合わせる。途中に Left があれば失敗する。

18.5.3 Monad

```
Right 5 >=> safeRecipE
-- Right 0.2

Right 0 >=> safeRecipE
-- Left "reciprocal of zero"

Left "previous error" >=> safeRecipE
-- Left "previous error"
```

成功値を次の計算に渡し、失敗ならそのまま伝播する。

19 Monad を特別視しすぎない

19.1 Monad は「文脈付き計算の接続」である

Monad は難解なものとして説明されがちである。しかし、少なくとも Maybe や Either から入る場合、かなり素直に理解できる。

Monad は、前の文脈付き計算の結果を見て、次の文脈付き計算を選ぶための仕組みである。

Maybe では、失敗するかもしれない計算をつなぐ。

```
Maybe a -> (a -> Maybe b) -> Maybe b
```

Either e では、失敗理由を持つ計算をつなぐ。

```
Either e a -> (a -> Either e b) -> Either e b
```

したがって、Monad を理解するには、いきなり抽象的な説明から入るより、

```
Just 5 :: Maybe Int
```

の「一段持ち上がった値」をどう計算に使うか、という問題から始めるとよい。

19.2 return と pure

古い Haskell の教材では、Monad に対して return がよく出てくる。

```
return :: Monad m => a -> m a
```

これは、普通の値をモナドの文脈に持ち上げる関数である。

しかし現在は、Applicative の pure を使う方が一般的である。

```
pure :: Applicative f => a -> f a
```

Maybe では、

```
pure 5
-- Just 5
```

である。

したがって、do 記法の最後でも、

```
pure r
```

と書ける。

19.3 一段持ち上げる、一段の中で変換する、次の計算へ渡す

この節全体の流れを一言で整理すると、次のようになる。

操作	型	直感
Just	<code>a -> Maybe a</code>	値を一段持ち上げる
fmap	<code>(a -> b) -> f a -> f b</code>	一段の中身を変換する
<*>	<code>f (a -> b) -> f a -> f b</code>	一段の中の関数を一段の中の値に適用する
>>=	<code>m a -> (a -> m b) -> m b</code>	中身を次の文脈付き計算へ渡す

Just 5 :: Maybe Int の理解は、この流れの出発点である。そこから、

```
fmap (+1) (Just 5)
```

へ進み、さらに

```
(+) <$> Just 3 <*> Just 4
```

へ進み、最後に

```
safeDiv 10 2 >>= safeRecip
```

へ進むと、Functor, Applicative, Monad はかなり自然につながって見える。

19.4 設計上の意味

この流れは、単なる抽象型クラスの学習ではない。プログラム設計上は、次の意味を持つ。

- Maybe は、失敗するかもしれない計算を型で明示する。
- Either は、失敗理由を持つ計算を型で明示する。
- Functor は、文脈を壊さず中身を変換する。
- Applicative は、独立した複数の文脈付き値を組み合わせる。
- Monad は、前段の結果に依存する文脈付き計算を順につなぐ。

したがって、Haskell におけるこれらの型クラスは、単に便利な抽象化ではなく、

計算がどのような文脈を持ち、その文脈をどのように保存・伝播・接続するかを型で表すための枠組み

である。

特に、Maybe と Either は、例外や null に頼らず、失敗可能性を型に現すための基本的な道具である。この意味で、Just 5 :: Maybe Int の「一段持ち上がっている」という感覚は、Haskell の抽象化を理解するための非常に重要な出発点である。

20 まとめ

ここまでの要点を整理する.

20.1 type

```
type C = Complex Double
```

は alias である.

C は Complex Double の別名であり, 新しい型ではない.

20.2 newtype

```
newtype UserId = UserId Int
```

は, 新しい型を定義する.

右辺の UserId は値コンストラクタであり,

```
UserId :: Int -> UserId
```

である.

newtype は, 1 コンストラクタ・1 フィールドに限定される. その代わり, 型安全性と実行時効率を両立できる.

20.3 data

```
data Point = Point Double Double
```

は直積型であり,

$$\text{Point} \cong \text{Double} \times \text{Double}$$

と見られる.

また,

```
data Shape
  = Circle Double
  | Rectangle Double Double
```

は直和型と直積型の組み合わせであり,

$$\text{Shape} \cong \text{Double} + (\text{Double} \times \text{Double})$$

と読める.

20.4 Maybe と Just

```
data Maybe a
  = Nothing
  | Just a
```

において,

```
Just :: a -> Maybe a
```

である.

したがって,

```
5      :: Int
Just 5 :: Maybe Int
```

であり, `Just 5` は 5 そのものではない.

`Just` は値を `Maybe` という文脈に一段持ち上げるコンストラクタである.

$$\text{Maybe } a \cong 1 + a$$

という見方ができる.

20.5 全体像

構文	役割	典型的な用途
<code>type</code>	型の別名	長い型を短くする, 意味を補う
<code>newtype</code>	1 つの型を包んで新しい型にする	型安全な意味の境界を作る
<code>data</code>	直積・直和によるデータ型定義	構造や選択肢を表す

以上のように, Haskell の型定義は単なるデータ構造定義ではなく, プログラム中の意味の境界を明確にするための道具である. 特に, `newtype` と `data` を使い分けることで, 数学的対象やドメイン概念を型レベルで整理できる.